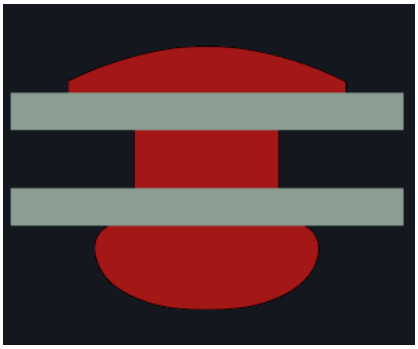
My SubTitle



date and time

Contents

Contents

rivt is an open source software program for writing and formatting engineering documents as text, HTML or PDF files (referred to as rivt docs). A *rivt doc* is generated from a rivt file (.py) that includes rivt markup and imports the rivtlib Python package. Multiple *rivt docs* may be organized and linked as a rivt report. This site is an example of a *rivt report*.

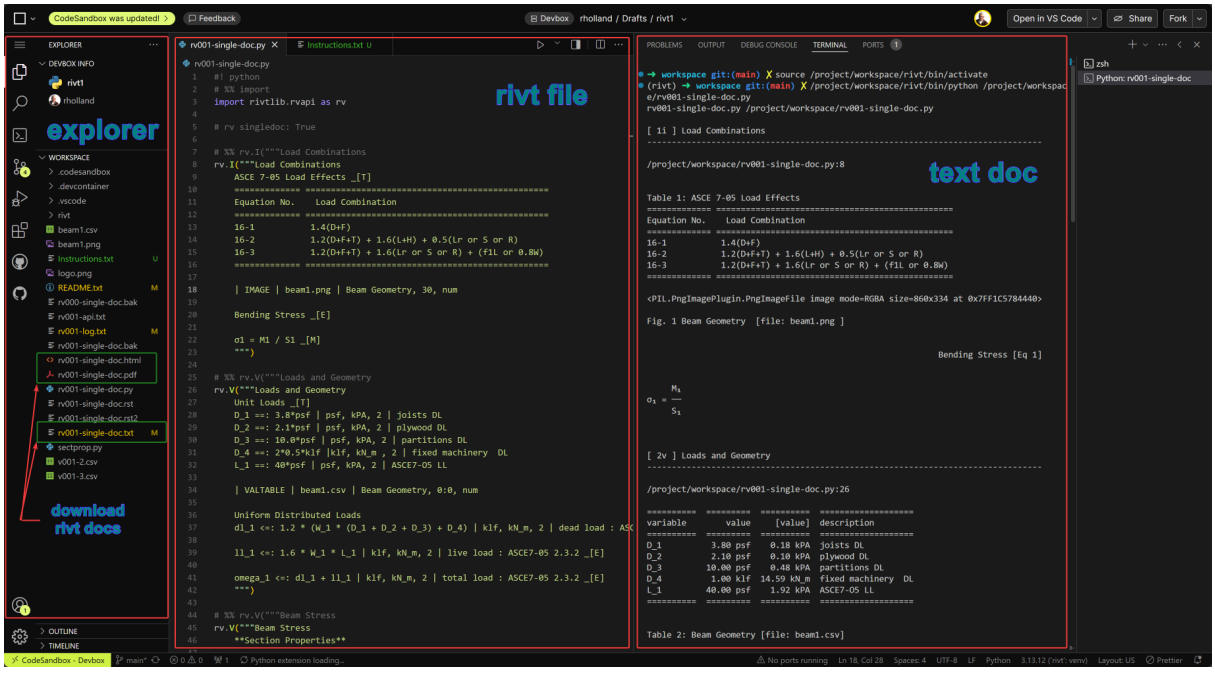
An interface for finding public *rivt files* on *GitHub* is here and a repository of open source models and calculations, including *rivt*, is here.

A.1 Start

[1t] Run examples

An overiew of *rivt* can be found here. This section reviews how to get started in writing *rivt files* and publishing *rivt docs*

rivt file examples can be run in the cloud or on a local computer. In both cases editing and publishing is typically carried out in an IDE. The rivt framework includes VSCode, but any IDE may be used. A typical IDE layout can be viewed at Codesandbox. The sandbox allows scrolling through an example rivt file (left panel) and text doc (right panel). Files may be downloaded from the file explorer (far left panel) including the text, pdf and html reports. Because the is unregistered editor files cannot be edited or processed.

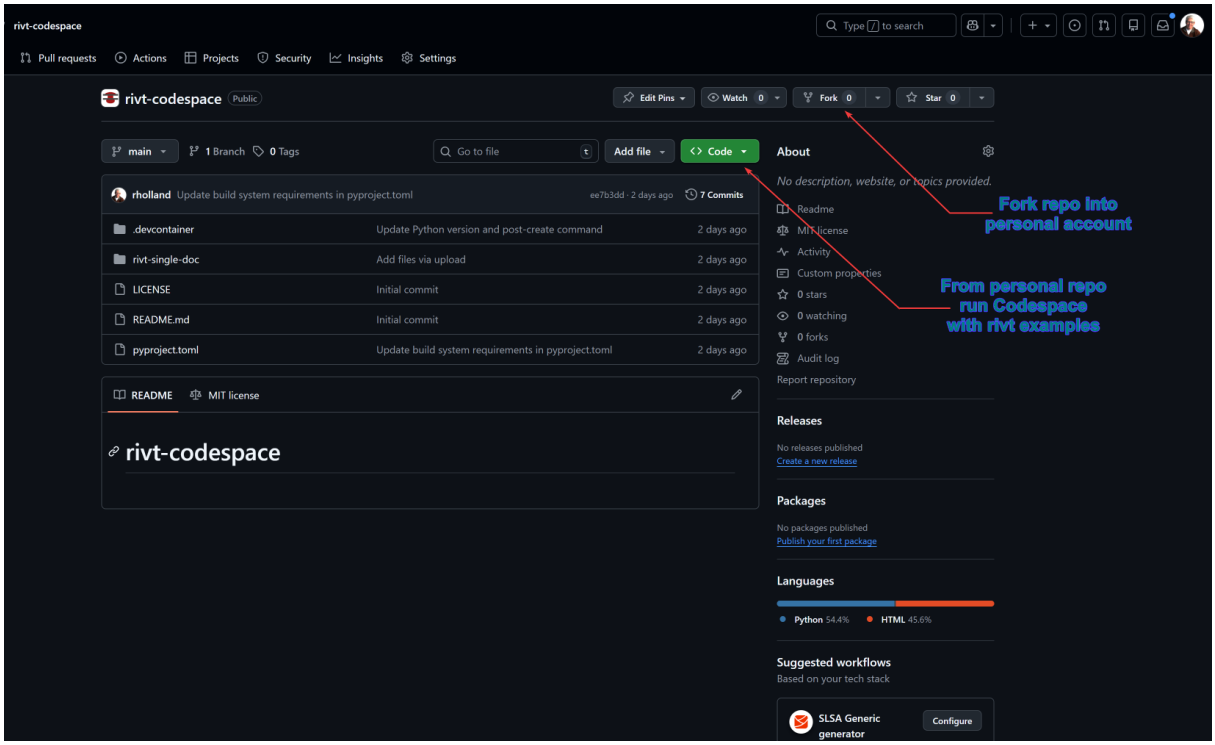


VSCode can be run locally or in the cloud. The cloud version is referred to as a Codespace . A rivt Codespace is a *VSCode* cloud environment with *rivt* extensions for editing and running *rivt files*. It can be forked (copied) and

run in a personal GithUb account that may be set up here. A rivt Codepsace with examples can be forked from this repository by following the steps below.

Fork a *rivt Codespace* in the cloud

```
1. Click the Fork button in the upper right corner.
    This creates a copy of the repository in your GitHub account.
2. Switch to your GitHub account and open the forked repository
3. Click the green Code button
4. Select the Codespaces tab
5. Click the rivt-codespace-examples option.
6. This creates a Codespace environment in your GitHub account
    and opens the repository in the online VSCode IDE. The environment
    includes the rivt packages and extensions needed to run and
    publish rivt docs and files.
```



Local installation methods

1. Windows portable zip folder ([2t] rivt-portable) *rivt-portable* is recommended for Windows users unfamiliar with Python.

2. System Python ([3t] rivt-sys with uv) with or without uv manager (rivt-uv)

[2t] rivt-portable

A *rivt-portable* installation is recommended for Windows users unfamiliar with Python. *rivt-portable* is a zip file :

```
win64-rivt-portable-n.n.n[an].zip
```

where n is a number representing the major, minor and patch release number. An *an* appended to the version is an *alpha* release where users should expect that features are missing and *rivt markup* syntax may change in the future. Zip file contents must be unzipped into a directory with read-write access - typically the users home folder or a flash drive.

The zipped and unzipped file sizes include the following contents and are approximately 1GB and 2GB respectively:

```
release: rivt-portable-1.0.0a4.zip

1. Python 3.14 with rivt packages
2. VSCode 1.109 with rivt extensions
3. rivt-1.1.0a4
4. Example rivt files
```

Releases may be download from the GitHub repository.

The primary advantages of *rivt-portable* are

1. simplified installation
2. package integration
3. isolation from system files.

After unzipping, VSCode and examples are started with a single shortcut.

The primary disadvantages of *rivt-portable* are that *rivt* packages and extensions must be updated by updating the zip file as a whole, and it is more difficult to integrate with other Python programs.

[3t] rivt-sys with uv

rivt may be installed at the system level.

1. Install Python using Python installers if not already installed. The minimum version required by *rivt* is Python 3.13.
2. Install *rivtlib* and dependencies using pip

```
pip install rivtlib
```

A list of the dependencies is here.

rivt-uv

Rather than installing *rivtlib* at the system level, it can be installed in an isolated environment using the uv package manager. The primary advantage of *rivt-uv* is the simplicity of updating packages while keeping them isolated from the system Python.

rivt-uv installation is recommended for users with some familiarity with Python and programming.

Step 1. Install uv

Different methods for installing *uv* are described here.

The recommended method for installing *uv* is from the command line:

Windows:

```
winget install --id=astral-sh.uv  -e
```

macOS and Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Step 2. Create the rivt environment

After installing *uv*, download and run the following rivt install script. It installs an isolated *rivt environment* and example files in a folder named *rivt-examples* in the users *Home* directory.

Windows: rivtuv.cmd

OSX and Linux: rivtuv.sh

Step 3. Run the example program from Pyzo.

The *rivt environment* is activated by:

Windows:

```
./venv/Scripts/activate
```

macOS and Linux:

```
source venv/bin/activate
```

rivt includes the Pyzo IDE for editing and running examples. Typing the command *pyzo-example* from the environment root will load the example file *rv00-simple-doc.py* where it can be edited and run.

The example file can also be run from the command line with the command:

```
python -m rivt rv001-single-doc.py
```

The uv environment can be completely removed with the following commands.

```
uv deactivate
rmdir /s /q rivt-examples
uv cache clean
```

[4t] rivt Framework

The full rivt framework may be integrated with rivt when it is installed at the system level, including uv. The framework includes:

1. *VSCode IDE <https://code.visualstudio.com/>* **and**
   extensions <https://marketplace.visualstudio.com/vscode>. *rivt* extensions and snippets are here.
2. *Git file version control <https://git-scm.com>*
3. GitHub file sharing <GitHub
4. *LaTeX document formatting <https://www.tug.org/texlive/>*
5. QCAD diagraming.

A.2 Motivations

[1t] Background

rivt is an open source software project that simplifies sharing and reuse of engineering documents. This has always been a challenge because the inclusion of text, images, tables, calculations, models and computer code required in reports has led to incompatible programs. Sharing and reuse has been restricted by the design and terms of use of existing software, including:

- documents written by different programs are incompatible
- frequent software updates are needed to maintain document access
- update costs are high
- newer document formats become inaccessible without upgrades
- software is limited to specific platforms
- document version control is limited
- report generation features are limited
- collaboration features are limited

The table below summarizes and compares limitations between different software programs. *rivt* is designed to address these limitations and function as a complement or replacement to existing software.

Software Comparison

Program	Rep [1]	Ver [2]	Txt [3]	Comp [4]	CP [5]	Collab [6]
Matlab	no	no	no	no	no	no
Mathcad	no	no	no	no	no	no
Mathematica	no	no	no	no	no	no
Cloud SaaS	limited	no	no	no	yes	limited
Excel	limited	no	no	yes	no	yes
Jupyter	no	no	no	yes	yes	yes
rivt	**yes**	**yes**	**yes**	**yes**	**yes**	**yes**

1 Report generation

2 Native version control

3 Plain text, readable input files

[2t] Use Cases

The primary use case for *rivt* is producing engineering documents that lie somewhere between back of envelope notes and calculations, and formal journal publications. In other words, it produces formatted, organized documents that are easy to edit.

The second use case is when flexibilty is needed to produce documents in a variety of formats including text, PDF or HTML.

rivt files can function as a front and back end for:

1. software control
2. visualization
3. instrumentation

rivt docs can be used for:

1. internal communication
2. research documentation
3. government permits
4. technical reports
5. funding applications

Because *rivt* is compatible with collaborative tools it may be used in:

1. teaching
2. for presentations
3. real time collaboration

A.3 FAQ

[1t] Contact

support@rivt.info

[2t] Licenses

What is the open source license for *rivtlib*?

rivtlib is distributed under the MIT open source license.

Which open source licenses applies to *rivt*?

The Python packages that are part of the *rivt project* are distributed primarly under the BSD, Appache and MIT open source licenses. Python itself is distributed under the Python Software Foundation License (PSFL).

Which open source licenses apply to *rivt files*?

The open source license for a public rivt file is specified in the _[[Metadata]] block tag. The default license is the MIT license.

Which open source licenses apply to the *rivt framework*?

VSCode - Visual Studio Code uses a proprietary Microsoft license for the compiled product, allowing free use for development, testing and commercial applications. The source code ("Code - OSS") is under the MIT license. You can use VS Code freely for any purpose, including commercial projects, but some bundled extensions or specialized components (connections to the Microsoft servers and marketplace) may have different terms. Most extentions in the Marketplace use the MIT license.

VSCodium is a fully open source alternative that removes Microsoft's proprietary components and built-in telemetry. However its extension marketplace is less complete.

LaTeX - LaTeX is released under the LaTeX Project Public License (LPPL) for LaTeX components and MIT and BSD variants for many core utilities and packages.

QCAD - The QCAD source code is distributed under the open source license GPLv3 with exceptions to allow proprietary extensions.

Git - The Git software is released under the GNU General Public License Version 2.0 (GPLv2).

[3t] *rivt* design and development

What is the *rivt* release schedule?

1.0.x bug releases are issued as needed.

1.x.0 minor releases are planned monthly following the release of version 1.0.0

X.0.0. major releases will have backward compatiblity. See past releases and changelog .

What are the programs and libraries used to build *rivt*?

The *rivt project* uses dozens of Python libraries (see [6t] Project requirements):

- numpy, sympy and related scientific computing libraries
- docutils and related text processing libraries
- Reportlab, rst2pdf and related PDF generation libraries
- Sphinx, Pydata and related HTML libraries

The *rivt framework* uses the following open source and free programs:

- VSCode for editing
- LaTeX for PDF generation
- QCAD for diagramming
- Git for document sharing and version control

B.1 Overview

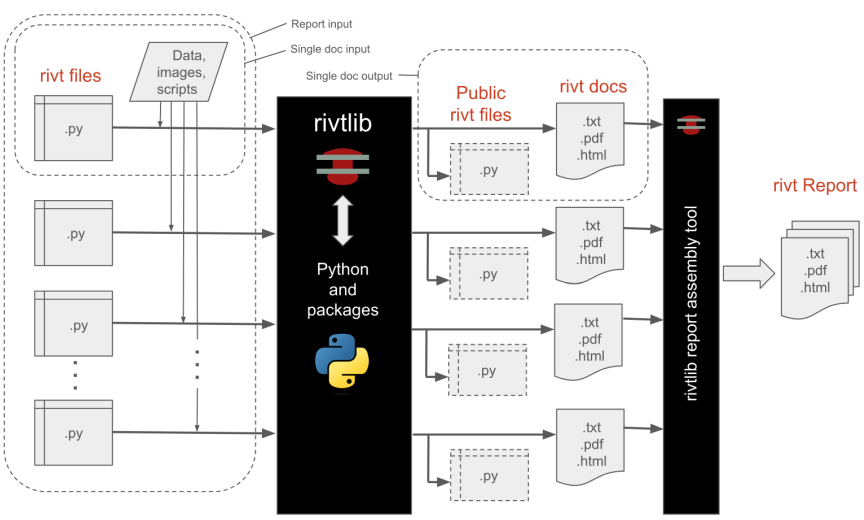
[1t] Summary

rivt is an open source Python project that imports the rivtlib Python package and dependencies ([6t] Project requirements). The lightweight Pyzo IDE is also installed for editing and publishing *rivt file* examples. The VSCode IDE is a more full featured IDE that is part of the *rivt framework* and installed separately.

A rivt file publishes a formatted rivt doc as a text, PDF or HTML file. A *rivt file* is a Python file (.py) that imports the rivtlib Python package and includes rivt markup. A collection of *rivt docs* may be collated as a rivt report.

rivt file examples are provided here. An interface for searching *public rivt files* on *GitHub* is here. A *public rivt file* is all or parts of a *rivt file* the author chooses to share under an Open Source license. *rivt* itself is distributed under the MIT open source license. (see [2t] Licenses).

[2t] rivt docs



rivt Doc Processing

Each rivt file outputs a corresponding doc in the format specified in *rv.D*. A doc number has the form:

```
rvAnn-filename.py
```

where rv is a required prefix, A is an alphanumeric character and nn are two digit non-negative integers.

A *rivt report* is organized using the rivt *doc numbers*. If the *rivt file* names are:

```
rvA01-filename.py
rv105-filename.py
rv212-filename.py
```

the *report numbers* would be:

- A.1 (division A, subdivision 1)
- 1.5 (division 1, subdivision 5)
- 2.12 (division 2, subdivision 12)

Note that leading zeroes are dropped. *Docs* are sorted alpha-numerically into divisions and subdivisions in the *report*.

Docs are assembled from sources and published using a specified folder structure. The *singledocB* variable overrides the default report structure and specifies that resource files and *docs* are read from and written to

the *rivt file* folder. It is intended for simpler, standalone *docs* with more limited format control. The comment variable is specified immediately after the import statement:

```
import rivtlib.rvapi as rv

# rv singledocB = True
```

The default setting is False.

[2t] rivt API

The *rivt API* includes API functions, [3t] Markup and files. The API is designed to be:

- **lightweight**
  rivt markup is made up of about 3 dozen tags and commands, and wraps reStructuredText.
- **flexible**
  A *rivt file* produces a text, HTML or PDF *doc*. Multiple *docs* can be organized into a report.
- **extensible**
  rivtlib is written in Python with direct access to thousands of Python packages.

[3t] Markup

An API function starts in the first column and takes a triple quoted rivt string (rS) argument. The first line of a *rivt string* is a header substring, followed by a content substring indented 4 spaces for improved readability and section folding.

The *header substring* specifies the section title and other processing parameters. The content substring includes rivt markup and other arbitrary text. For further details see here.

```
rv._("""Section Label | parameters

     Content text that is
     indented four spaces.

    ...

    """)
```

[4t] Files/Folders

A rivt file is a Python plain text file (*.py*) that includes *rivt markup* and imports the rivtlib package and API into the *rv* namespace:

```
import rivtlib.rvapi as rv
```

rivt files are stored in a root folder designated with the prefix *rivt-*. Each *rivt file* and corresponding *rivt doc* has a prefix that is used to organize the report. The prefix has the form

```
rvANN-filename.py
```

where A is an alpha-numeric character used for organizing report divisions, and NN is a two digit number used to organize sub-divisions. The file name is used as the *doc* title unless overridden in *rv.D*. Use underscores or hyphens rather than spaces to seprate words in the file name. They are replaced with spaces when a *doc* title is extracted.

The top level folder structure is shown below. A more detailed description of the folder structure is here.

```
[rivt-]Report-Label/                Report Folder
    ├── [rv101-]filename1.py        rivt file
    ├── [rv102-]filename2.py        rivt file
    ├── [rv201-]filename3.py        rivt file
    ├── [rv202-]filename4.py        rivt file
    ...
    ├── [.vscode]/                  optional VSCode settings
    ├── [_public]/                  public rivt files
    ├── [_publish]/                 published docs and reports
    ├── [_stored]/                  stored files
    ├── [Rst_docs]/                 restructured text files
    ├── [Src]/                      source files
    └── README.txt                  searchable text report
```

B.1 Framework

[1t] Install *rivt-framework*

The *rivt framework* includes installation of the following programs:

1. VSCode and VSCode extensions. *rivt* extensions and snippets are here.
2. Git and set up a GitHub
3. LaTeX
4. FreeCad

C.1 Markup

[1t] API functions

rivt has seven API functions. The name *rivt* is an acronym taken from the four functions that process content. The remaining three functions are used for document generation and debugging.

API Function	Name	Purpose
rv.R (rS)	Run	Run shell commands
rv.I (rS)	Insert	Insert static sources
rv.V (rS)	Values	Calculate values
rv.T (rS)	Tools	Python and markup scripts
rv.D (rS)	Doc	Publish docs

API Function	Name	Purpose
rv.S (rS)	Skip	Skip section (comments and debugging)
rv.X (rS)	Exit	Exit rivt (debugging)

API functions define doc sections. If interactive IDEs like *VSCode* or *Spyder* are used to edit and run *rivt files*, functions can be processed individually as cells by using the standard notebook hash notation:

```
# %% optional label
rv.API_FUNCTION("""rivt string""")
```

[2t] rivt string

Each API function takes a triple quoted rivt string argument composed of a line header substring line followed by a multiple-line content substring.

The *header substring* defines section processing parameters. The *content substring* includes rivt markup and is indented four spaces for improved readability and navigation (e.g. section folding).

Header substring

The header substring starts with a *section label* that is used as the section title, followed by a vertical bar and three comma separated *section parameters* that override default behavior.

```
rv._("""Section Label | doc;stored, private;public, section;merge

     Content substring indented 4 spaces

    ...

    """)
```

The parameters include the following, in any order:

private/public

Determines whether the API section text is copied to the the *Public* folder *rivt file* for sharing.

doc/stored

Determines whether the *rivt string* is formatted and printed in the doc, or just annotated in the doc and written to a file in the *Stored* folder for optional inclusion as an appendix.

section/merge

Determines whether the API starts a new *doc* section or is merged into the previous section.

Default settings in the *header substring* do not need to be specified. The default setting for each API is listed first (in bold) in the table below.

API	private;public	doc;stored	section;merge
rv.R	**private**;public	**stored**;doc	**merge**;section
rv.I	**private**;public	**doc**;stored	**section**;merge

API	private;public	doc;stored	section;merge
rv.V	**private**;public	**doc**;stored	**section**;merge
rv.T	**private**;public	**stored**;doc	**merge**;section
rv.S	**private**;public	**stored**	**merge**
rv.D	**public**	**stored**	**merge**
rv.X	•	•	•

Examples of *header substring* settings are shown below.

- An example with explicit defaults (they do not have to be declared).

```
# This

rv.I("""A New Section | private, doc, section

    Content text
    ...

    """)

# is equivalent to:

rv.I("""A New Section

    Content text

    ...

    """)
```

- An example that merges a section into the previous section.

```
rv.I("""A Merged Section | merge

    Content text

    ...

    """)
```

Content substring

The content substring is indented four spaces for legibility and code folding. It includes line tags, block tags and commands along with text.

```
rv._("""Section Label

    Content text indented 4 spaces.
    ...

    """)
```

Content is converted line by line into formatted text and RestructuredText, and then further processed into HTML or PDF. If a line does not contain a *command* or *tag* it is passed through as is, which allows for the *Insert* function (rv.I) including some *restructured text* directly i.e. surrounding words with * or ** will format a word as italic or bold respectively.

In addition block tags in the *Tools function* (rv.T) directly supports processing HTML, LaTeX and reStructuredText scripts.

[3t] Tags and Commands

Line Tags

A line tag formats a line of text and is denoted with a single **_[LETTER]**, placed at or near the end of the line, depending on the tag.

Block Tags

A block tag formats a block of text and begins with **_[[TAGNAME]]** and terminates with **_[[END]]**.

Commands

rivt commands read and write external files. They typically start in the first column with a vertical bar (|) followed by the command name, file path, and parameters.

The exceptions are the definition (**==:**), assignment (**<=:**), function (**:=:**) and compare (**<>**) commands that are used to define, assign and compare values.

```
| COMMAND | relative path | parameters
```

File paths are specified relative to the *rivt root folder*. The *rivt report* folder structure is described here..

If the path is ommitted the default path for each command is applied. If the *singledoc* parameter is set, the *resource files* and *docs* are stored in the *rivt root folder*.

Tag and Command syntax for each API type is defined and described using the following format:

[4t] Folders

For single docs the folder structure is the same as a *report* folder but without the *Src* and *_stored* folders. The files nomrally stored there are stored in the root folder. The *report script* is not used for single docs.

The top level folder structure is shown below. A more detailed description of the folder structure is here.

```
[rivt-]Report-Label/                Report Folder
    ├── [rv101-]filename1.py        rivt file
    ├── [rv102-]filename2.py        rivt file
    ├── [rv201-]filename3.py        rivt file
    ├── [rv202-]filename4.py        rivt file
    ...
    ├── [.vscode]/                  optional VSCode settings
    ├── [_public]/                  public rivt files
    ├── [_publish]/                 published docs and reports
    ├── [_stored]/                  stored files
    ├── [Rst_docs]/                 restructured text files
    ├── [Src]/                      source files
    └── README.txt                  searchable text report
```

C.2 Line Tags

[1t] Line Tag Summary

Format a line of text

API Scope	Line Tag	Description
rv.l	text **_[C]**	[2t] Center text
rv.l	text **_[R]**	[3t] Right justify text
rv.l	**text math _[M]**	[4t] Text math
rv.l	**LaTeX math _[L]**	[5t] LaTeX math
rv.l	text **_[#]** text	[6t] Endnote number
rv.l	text **_[G] glossary term |** text	[7t] Glossary link
rv.l	text **_[S] label, section |** text	[8t] Section link
rv.l	text **_[D] label, file name |** text	[9t] Doc link
rv.l	text **_[U] label, url |** text	[10t] URL link
rv.V, l	text **_[V] var_name |** text	[11t] Variable value
rv.V, l	**text _[E]**	[12t] Equation number
rv.V, l	**title _[T]**	[13t] Table number
rv.V, l	**caption _[F]**	[14t] Figure number
rv.V, l	**_[P]**	[15t] New Page
rv.V, l	**##** text	nonprinting comment

[2t] Center text

Center line of text within the page margins.

_[C]

```
Syntax:
    text _[C]

Example:
    This text wil be centered. _[C]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[3t] Right justify text

Right justify line of text within the page margins.

_[R]

```
Syntax:
    text _[R]

Example:
    This text wil be right justified _[R]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[4t] Text math

Format math expression into text.

_[M]

```
Syntax:
    text math expression  _[M]

Example:
    f(x,y) = sin(x)**2 + y/5 _[M]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[5t] LaTeX math

Format LaTeX math expression.

_[L]

```
Syntax:
    LaTeX math expression  _[L]

Example:
    \frac{1}{\sqrt{x}} _[L]
```

API Scope	Insert
Doc Types	PDF, HTML

[6t] Endnote number

Assign endnote number to the text in order of processing. Endnotes are defined with the block tag _[[ENDNOTE]] and are listed at the end of the *doc*.

_[#]

```
Syntax:
    text _[#] more text

Example:
    This is a sentence with an endnote _[#] tag.
```

API Scope	Insert
Doc Types	text, PDF, HTML

[7t] Glossary link

Link a term to the glossary.

_[G]

```
Syntax:
    text _[G] glossary term

Example:
    A definition of _[G] namespace
    is provided in the glossary.
```

API Scope	Insert
Doc Types	PDF, HTML

[8t] Section link

Create a link to the section label defined in the API header with optional link text. If the text is ommitted the section label is used for the link.

_[S]

Syntax:

text _[S] link text, section title

Example:

This creates a link to _[S] Section Title, actual section label which provides more detail.

API Scope	Insert
Doc Types	PDF, HTML

[9t] Doc link

Link to a *doc* in a *report* with an optional link text. If the link text is ommitted the *doc* title will be inserted as the link term.

_[D]

```
Syntax:
    text _[D] text, rivt_file

Example:
    This is a link to _[D] rv101-filename1
    for reference.
```

API Scope	Insert
Doc Types	PDF, HTML

[10t] URL link

Link to a an external site with optional link text. If the link text is ommitted the url will be inserted as the link term.

_[U]

```
Syntax:
    text _[U] link label ,external url

Example:
    text _[U]  github, https://www.github.com
```

API Scope	Insert
Doc Types	PDF, HTML

[11t] Variable value

Insert the value of var_name _[V] in the sentence.

_[V]

```
Syntax:
    text var_name _[V] more text

Example:
    The value of my_var is my_var _[V].
```

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[12t] Equation number

Assign equation number to a line of text.

_[E]

```
Syntax:
    math text or label _[E]

    assignment command _[E]

Example:
    3x * 4/(1+n) _[E]

    x <= 3*IN + 4.1*FT | inch, cm, 2 | example 1 _[E]
```

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[13t] Table number

_[T]

```
Syntax:
    Table Title  _[T]

Example:
    A New Table _[T]
```

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[14t] Figure number

_[F]

```
Syntax:
    Figure caption _[F]

Example:
    Stress Distribution _[F]
```

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[15t] New Page

_[P]

```
Syntax:
    text _[P]

Example:
    Preceded by text or stand alone. _[P]
    _[P]
```

API Scope	Insert, Values
Doc Types	text, PDF, HTML

C.3 Block Tags

[1t] Block Tag Summary

Format blocks of text

API Scope	Block Tag	Description
rv.R	**_[[SHELL]]** process parameters	[2t] Shell script
rv.I	**_[[INDENT]]** spaces (4 default)	[3t] Indent text
rv.I	**_[[ITALIC]]** spaces (4 default)	[4t] Indent italic
rv.I	**_[[ENDNOTES]]** optional label	[5t] Endnotes
rv.I	**_[[TEXT]]** optional language	[6t] Text
rv.I	**_[[TOPIC]]** topic	[7t] Topic
rv.I	**_[[BOX]]** label	[8t] Box
rv.V,T	**_[[PYTHON]]** namespace	[9t] Python
rv.T	**_[[MARKUP]]** type	[10t] Markup block
rv.D	**_[[METADATA]]** label	[11t] Metadata
rv.D	**_[[LAYOUT]]** label	[12t] Layout block
all	**_[[END]]**	[13t] End block

[2t] Shell script

Runs shell scripts that run external programs. The shell parameters include specifying the operating system, process control and terminal window control. The os parameter specifies the terminal type. The wait parameter specifies whether rivt file processing waits for the script to complete before continuing. The open parameter specifies whether to keep the shell window open after execution.

_[[SHELL]]

```
Syntax:
    _[[SHELL]] win;mac;linux,wait; nowait,open; close
    shell command
    ...
    _[[END]]


Example:
    _[[SHELL]] win,nowait,open
    dir
    path
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[3t] Indent text

Indents text block the specified number of spaces.

_[[INDENT]]

```
Syntax:
    _[[INDENT]] number of spaces
    text
    text
    ...
    _[[END]]

Example:
    _[[INDENT]] 8
    This is a sentence that will be
    indented 8 spaces.
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[4t] Indent italic

_[[ITALIC]]

Italicizes and indents the text block the specified number of spaces.

```
Syntax:
    _[[ITALIC]] number of spaces
    text
    ...
    _[[END]]

Example:
    _[[ITALIC]] 4
    This is a sentence that will be
    italicized and indented 4 spaces.
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[5t] Endnotes

_[[ENDNOTES]]

Assigns numbers and formats endnotes in order of processing. Each endnote is separated by a blank line and is numbered in order of occurrence.

```
Syntax:
    _[[ENDNOTES]] label
    Endnote 1.

    Endnote 2.
    ...
    _[[END]]

Example:
    _[[ENDNOTES]] ACI citations
    This endnote is assigned to the first endnote tag (_[#]) in order of
    of processing.

    This second endnote is separated by a blank line and assigned
    to the second

    This is a third endnote.
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[6t] Text

_[[TEXT]]

Reads and formats text and code. The language parameter specifies formatting and syntax coloring. Languages include:

- *literal*
- *python*
- *text*
- *sh*
- *cmd*

```
Syntax:
    _[[TEXT]] language
    text and code
    ...
    _[[END]]

Example:
    _[[TEXT]] python
    # some code
    print(1)
    a = 1 + 3
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[7t] Topic

_[[TOPIC]]

Formats a topic block.

```
Syntax:
    _[[TOPIC]] topic title
    text
    ...
    _[[END]]

Example:
```

```
    _[[TOPIC]] topic title
    Text related to the topic.
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[8t] Box

_[[BOX]]

Draws a box around the block of text.

```
Syntax:
    _[[BOX]] optional label
    text
    ...
    _[[END]]

Example:
    _[[BOX]]
    This is a sentence.
    A second sentence.
    A box is drawn around the sentences.
    _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[9t] Python

_[[PYTHON]]

Executes Python script in the *rivt namespace* or a user specified namespace. File paths in the script are relative to the *rivt file* folder.

```
Syntax:
    _[[Python]] topic title
    code
    ...
    _[[END]]

Example:
    _[[PYTHON]] *rvspace*; user namespace
```

```
    b = 1*inch + 2.2*ft
    print(b)
    _[[END]]
```

API Scope	Value, Tool
Doc Types	text, PDF, HTML

[10t] Markup block

Inserts HTML into an HTML *doc*, LaTeX into a PDF *doc*, and reStructuredText into either PDF or HTML.

_[[MARKUP]]

```
Syntax:
    _[[MARKUP]] *html;latex;rst*
    markup
    ...
    _[[END]]

Example:
    _[[MARKUP]] rst
    This is **bold** and
    this is *italic* in
    reStructuredtext
    ...
    _[[END]]
```

API Scope	Tool
Doc Types	text, PDF, HTML

[11t] Metadata

_[[METADATA]]

Metadata is written to the *rvAnn-apilog.py* file and is converted internally to a Python dictionary rvmetaD with key name taken from the label e.g. *docnameS* or *authorsD*.

```
Synatax - defaults:
    _[[METADATA]] optional label
    docname = ""
    authors = ""
    version = 0.0.0
    email = ""
    repo = ""
```

```
    license = https://opensource.org/license/mit
    fork1 = author, version, email, repo
    fork2 = author, version, email, repo
    _[[END]]

Example:
    _[[METADATA]]
    docname = Bearing Pressures
    authors = rholland, rward
    version = 0.1.1
    email = rholland@email.com
    repo = https://github.com/rivt-info/rivt-simple-doc
    license = https://opensource.org/license/mit
    _[[END]]
```

API Scope	Docs
Doc Types	text, PDF, HTML

[12t] Layout block

Overrides default layout settings.

_[[LAYOUT]]

```
Syntax - defaults:
    _[[LAYOUT]] optional label
    variables
    ...
    _[[END]]

Example:
    _[[LAYOUT]]
    docname = ""
    footer = date, docname, page
    _[[END]]
```

API Scope	Doc
Doc Types	text, PDF, HTML

[13t] End block

_[[END]]

Terminates a block tag.

```
Syntax:
    _[[NEWPAGE]] optional label

Example:
    _[[END]]
```

API Scope	All
Doc Types	text, PDF, HTML

C.4 Commands

[1t] Command Summary

format files and equations

API Scope	Command	Description
rv.R	**| SHELL |** rel path | os, wait	[2t] Shell file
rv.I	**| TEXT |** rel path | language	[3t] Text file
rv.V, I	**| TABLE |** rel path | title, width, rows, align, head	[4t] Table file
rv.V, I	**| IMAGE |** rel path | caption, scale, number	[5t] Image file
rv.V, I	**| IMAGE2 |** rel pth1, rel pth2 | cap1,cap2,sca1,sca2,num	[6t] Adjacent images
rv.V	**| VALTABLE |** rel path | title, rows, number	[7t] Valtable file
rv.V	a **==:** 1*IN | unit1, unit2, decimal | label	[8t] Define value
rv.V	c **<=:** expression | unit1, unit2, decimal | label	[9t] Assign value
rv.V	c **:=:** expression | unit1, unit2, decimal | label	[9t] Function value
rv.V	a **<** c | decimal | text1, text2, color1, color2	[10t] Compare values
rv.T, V	**| PYTHON |** rel path | namespace	[11t] Python file
rv.T	**| MARKUP |** rel path | type	[12t] Markup file
rv.D	**| ATTACHPDF |** rel path | place, title	[13t] Attach PDF
rv.D	**| PUBLISH |** doc title | type	[14t] Publish doc

Default command paths

See here for the folder structure. If files are in the default path only the file name needs to be provided.

Command	Default Path
| SHELL |	**/Files/Scripts/**
| TEXT |	**/Files/Data/**
| TABLE |	**/Files/Data/**
| IMAGE |	**/Files/Image/**
| IMAGE2 |	**/Files/Image/**
| VALTABLE |	**/Files/Data/** [1]
| PYTHON |	**/Files/Scripts/**
| MARKUP |	**/Files/Scripts/**
| ATTACHPDF |	**/Files/Attach/**
| PUBLISH |	depends on file type

[1] use /stored/data/filename to read values defined in the rivt file

[2t] Shell file

The SHELL command runs shell scripts including .cmd, .bat and .sh files. The *os* parameter specifies the operating system: *win*, *mac* or *linux*. The *wait; nowait* specifies whether rivt file processing waits for the script to complete before continuing.

If the *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/run/* . Otherwise the path is specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

```
Syntax:
    | SHELL | relative file path | win;mac;linux,wait;nowait

Example:
    | SHELL | file1.cmd | win, nowait
```

API Scope	rv.R
File Types	.cmd, .bat, .sh, .bsh
Doc Types	text, PDF, HTML

[3t] Text file

The TEXT command reads and formats text and code files. The language parameter specifies formatting and syntax coloring. Language types include:

- *literal*

- *python*
- *text*
- *sh*
- *cmd*
- *reStructuredText*

The *literal* type inserts text into the *doc* without formatting.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/data/* . Otherwise the path needs to be specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

```
Syntax:
    | TEXT | relative file path | language - see list above

Example:
    | TEXT | file1.txt | literal
```

API Scope	rv.l
File Types	txt, .py, .cmd, .bat, .sh, .rst
Doc Types	text, PDF, HTML

[4t] Table file

The TABLE command reads csv, xls, and rst files and outputs formatted tables. The title may be ommited by inserting a hyphen "-". The width parameter specifies the maximum character width of a column. The align parameter specifies the cell justification - left, center, right. The number parameter specifies whether the table is numbered. For csv files, the head parameter specifies whether the first row is a column header.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/data/* . Otherwise the path needs to be specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

| TABLE |

```
Syntax:
    | TABLE | rel path | title, max width, rows, l;c;r, head;nohead, num;nonum

Example:
    | TABLE | file1.csv | Forces, 30, 0:0, c, nohead, num
```

API Scope	rv.l, rv.V
File Types	csv, xls, rst
Doc Types	text, PDF, HTML

[5t] Image file

The IMAGE command reads a PNG or JPEG file and centers it in the *doc*. The scale parameter is a decimal fraction of the page width. The caption may be ommited by using a single hyphen. The *num;nonum* parameter specifies whether to assign a figure number. The image path is inserted in the text *doc* instead of the image.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/img/* . Otherwise the path needs to be specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

```
Syntax:
    | IMAGE | relative path | caption, scale, num;nonum

Example:
    | IMAGE | file1.png | Map, 0.5, num
```

API Scope	rv.V, rv.I
File Types	PNG, JPG
Doc Types	PDF, HTML

[6t] Adjacent images

The IMAGE2 command reads two PNG or JPEG file and places them side by side in the *doc*. The scale parameters are a decimal fraction of the page width. The captions may be ommited by using a single hyphen for either or both images. The *fig;nofig* parameters specify whether to assign a figure number to either or both images. The image path is inserted in the text *doc* instead of the image.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/img/* . Otherwise the path needs to be specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

```
Syntax:
    | IMAGE2 | relative path | cap1, cap2, scale1, scale2, num;nonum, num;nonum

Example:
    | IMAGE2 | file1.png, file2.png | Map, Photo, .5,.5, num, num
```

API Scope	rv.I, rv.V
File Types	PNG, JPG jenn
Doc Types	PDF, HTML

[7t] Valtable file

The VALTABLE command imports and defines values from a *csv* or *xls* file. The file format is:

```
var = value, unit1, unit2, decimal, label
```

The path variable is either from *src* or *stored* depending on if the values were generated by rivtlib or provided by the author. The title parameter is the table title. A hyphen omits the title. The *rows* parameter specifies the rows to import. The *num; nonum* parameter specifies whether to assign a table number to the values table.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/vals/* . Otherwise the path needs to be specified relative to the report root (rivt file folder). If the values are read from prior calculated values, they will be found in the */stored/vals* folder. If the doc is a single doc the file is read from the rivt file folder.

```
Syntax:
    | VALTABLE | relative path | title, rows, *num;nonum*

Example:
    If read from the default folder:
    | VALTABLE | newvals.csv | Beam Properties, 1:10, num
    If read from the stored folder:
    | VALTABLE | /stored/vals/vA01-2.csv | Beam Properties, 1-10, num
```

API Scope	rv.V
File Types	.csv, .xls,
Doc Types	text, PDF, HTML

[8t] Define value

Defines a value and writes it to the file *vdocnum-s.csv* where *num* is the *docnumber* and *s* is the section number. The file is written to the folder *stored/vals* unless *singledocB* is set to *True* in the comment variable.

The stored values can read and defined in other rivt files using the VALUES command.

```
Syntax:
    c =: 5*unit | unit1, unit2, decimals | label, *num,nonum*

Example:
    D_1 =: 10*IN | IN, M, 3 | beam depth, num
```

API Scope	rv.V
File Types	.csv
Doc Types	text, PDF, HTML

[9t] Assign value

Assigns a value to an equation and writes the values to a file *vdocnum-s.csv* where *num* is the *doc number* and *s* is the section number. The file is written to the folder *stored/vals* unless *rv single doc* is set to *True* in which case values are stored in the rivt file folder (root).

The label is a reference printed with the equation. The units specify the result expressed in two different units. The integer specifies the number of places after the decimals.

```
Syntax:
    b <=: a * 10*FT | unit1, unit2, decimals | label, num;nonum

Example:
    b_1 <=: E_1 * 12.1*IN^2 | KIP, KN, 2 | Std. 123, num
```

API Scope	rv.V
File Types	.csv
Doc Types	text, PDF, HTML

[9t] Function value

Assigns a value to an equation and writes the values to a file *vdocnum-s.csv* where *num* is the *doc number* and *s* is the section number. The file is written to the folder *stored/vals* unless *rv single doc* is set to *True* in which case values are stored in the rivt file folder (root).

The label is a reference printed with the equation. The units specify the result expressed in two different units. The integer specifies the number of places after the decimals.

```
Syntax:
    c :=: function name() | unit1, unit2, decimals | label, num;nonum

Example:

    c_1 :=: func1(a,b) | KIP, KN, 2 | ACI 318-19 Table 22.5.5.1, num
```

API Scope	rv.V
File Types	.csv
Doc Types	text, PDF, HTML

[10t] Compare values

Inserts a text with alignment and color on the following line if the expression evaluates to true. text1 and text2 are printed if true or false repectively. Colors are assigned accordingly as *red;blue;yellow;green;black;gray* defined in the *style files*. The text is right aligned.

Comparison operators:

```
==    Equal x == y
!=    Not equal     x != y
>     Greater than  x > y
<     Less than     x < y
>=    Greater than or equal to x >= y
<=    Less than or equal to x <= y
```

<>

```
Syntax:
    a < b | text1, text2, color1, color2

Example:
   a < b |OK, NOT OK, blue, red
```

API Scope	rv.V
Doc Types	text, PDF, HTML

[11t] Python file

Executes Python code in the *rivt namespace* or user specified namespace. File paths used in the script are relative to the root *rivt* folder.

```
Syntax:
    | PYTHON | relative path | *rvspace*; user space

Example:
    | PYTHON | script1.py | rv-space
```

API Scope	rv.V, rv.T
File Types	.csv
Doc Types	text, PDF, HTML

[12t] Markup file

Inserts HTML into an HTML *doc*, LaTeX into a PDF *doc*, and reStructuredText into either PDF or HTML. LaTeX requires the installation of *texlive*.

| MARKUP |

```
Syntax:
    | MARKUP | relative path | *rst, html, latex*

Example:
    | MARKUP | script.rst | rst
```

API Scope	rv.t
File Types	html, rst, tex
Doc Types	text, PDF, HTML

[13t] Attach PDF

Appends or prepends a PDF file to the *doc*. The title parameter generates an Appendix cover page with the specified title. A "-" omits the over page. For HTML *docs* the file is inserted as a donwload link.

| ATTACHPDF |

```
Syntax:
    | ATTACHPDF | relative path | *front;back*, title

Example:
    | ATTACHPDF | relative path | *front;back*, title
```

API Scope	rv.D
File Types	tex, html, text, pdf
Doc Types	text, PDF, HTML

[14t] Publish doc

Publishes a *doc*.

```
Syntax:
    | PUBLISH | ini relative path | rst2pdf, texpdf, tex, hmtl

Example:
    | PUBLISH | ini relative path | rst2pdf
```

API Scope	rv.D
File Types	tex, html, text, pdf

API Scope	rv.D
Doc Types	text, PDF, HTML

C.5 Run - rv.R

[1i] rv.R Markup

The *Run* API function executes shell commands.

[2i] Block Tags

Block Tag	Description
_[[SHELL]] process parameters	[2t] Shell script
_[[END]]	[13t] End block

[3i] Commands

Command	Description
| SHELL | relative path | os, wait	[2t] Shell file

C.6 Insert - rv.I

[1i] rv.I Markup

The *Insert* API function inserts and formats static sources into the *doc*, including images, tables, links and formatted text.

[2i] Line Tags

Line Tag	Description
text **_[C]**	[2t] Center text
text **_[R]**	[3t] Right justify text
text math _[M]	[4t] Text math
LaTeX math _[L]	[5t] LaTeX math
text **_[#]** text	[6t] Endnote number
text **_[G] text, section label**	[8t] Section link

Line Tag	Description
text **_[D] text, rivt_file**	[9t] Doc link
text **_[U] text, label**	[10t] URL link
text **_[V] var_name** text	[11t] Variable value
assign var or **label _[E]**	[12t] Equation number
title _[T]	[13t] Table number
caption _[F]	[14t] Figure number

[3i] Block Tags

Block Tag	Description
_[[INDENT]] spaces (4 default)	[3t] Indent text
_[[ITALIC]] spaces (4 default)	[4t] Indent italic
_[[ENDNOTES]] optional label	[5t] Endnotes
_[[TEXT]] optional language	[6t] Text
_[[TOPIC]] topic	[7t] Topic
_[[END]]	[13t] End block

[4i] Commands

Command	Description
| IMAGE | relative path | scale, caption, figure	[5t] Image file
| IMAGE2 | rel path1, rel path2 | s1, s2, c1, c2, fig1, fig2	[6t] Adjacent images
| TEXT | relative path | language	[3t] Text file
| TABLE | rel path | title, width, rows, align, head	[4t] Table file

C.7 Values - rv.V

[1i] rv.V Markup

The *Values* API function defines values and evaluates equations and functions.

[2i] Line Tags

Format a line of text

Line Tag	Description
text **_[V] var_name** | text	[11t] Variable value
label _[E]	[12t] Equation number
title _[T]	[13t] Table number
caption _[F]	[14t] Figure number

[3i] Block Tags

Format or execute a block of text

Block Tag	Description
_[[PYTHON]] namespace	[9t] Python
_[[END]]	[13t] End block

[2i] Commands

Format files and calculations

Command	Description
TABLE | rel path | title,width,rows,align,head,num	[4t] Table file
IMAGE | relative path | scale, caption, figure	[5t] Image file
IMAGE2 | rel path1, rel path2 | c1, c2, s1, s2, f1, f2	[6t] Adjacent images
VALTABLE | rel path | title, width, rows	[7t] Valtable file
PYTHON | relative path | namespace	[11t] Python file
a **==:** 1*IN | unit1, unit2, decimal, num | label	[8t] Define value
c **<=:** expression | unit1, unit2, decimal, num | label	[9t] Assign value
c **:=:** expression | unit1, unit2, decimal | label, number	[9t] Function value

Command	Description
a **<** c | decimal | text1,text2,align, num	[10t] Compare values

C.8 Tool - rv.T

[1i] rv.T Markup

The *Tools* API function executes markup and scripts.

[2i] Block Tags

Block Tag	Description
_[[PYTHON]] namespace	[9t] Python
_[[MARKUP]] type	[10t] Markup block
_[[END]]	[13t] End block

[3i] Commands

Command	Description
| PYTHON | relative path | namespace	[11t] Python file
| MARKUP | relative path | type	[12t] Markup file

C.9 Doc - rv.D

[1i] rv.D Markup

The *Doc API* publishes formatted *docs* from the rivt API strings.

[2i] Block Tags

Block Tag	Description
_[[LAYOUT]] label	[12t] Layout block
_[[METADATA]] label	[11t] Metadata

[3i] Commands

Command	Description
| PDFATTACH | relative path | place, cover	[13t] Attach PDF
| PUBLISH | ini rel. path | type	[14t] Publish doc

D.1 Docs

[1t] rivt Files

Each rivt file outputs a corresponding formatted doc. A doc number has the form

```
rvAnn-filename.py
```

where rv is a required prefix, A is an alphanumeric character and nn are non-negative integers.

A *rivt report* is organized using the rivt *doc numbers*. If the *rivt file* names are:

```
rvA01-filename.py
rv105-filename.py
rv212-filename.py
```

the *report numbers* would be:

- A.1 (division A, subdivision 1)
- 1.5 (division 1, subdivision 5)
- 2.12 (division 2, subdivision 12)

Note that leading zeroes are dropped. *Docs* are sorted alpha-numerically into divisions and subdivisions in the *report*.

[2t] Docs and Reports

Each rivt file outputs a corresponding formatted doc written to the *publish* folder unless it is a single doc.

Docs may be text, HTML or PDF. PDF and HTML *doc* files are produced using using the *rst2pdf* and *Sphinx* Python libraries. The *report folder* structure for *docs* is described in [3t] Report Folders. Reports are assembled from by the *rivt-report.py* script in the *Rst_Docs* folder.

[3t] Single docs

A single doc that will not be part of a report may use a simplified folder structure. A single *doc* is formatted by setting the comment variable, immediately following the *rivtlib* import statement.

```
from rivtlib import rivtlib.rvapi as rv

# rv singledoc=True
```

The folder structure for *single docs* is simmilar to the structure for *report* folders described in [3t] Report Folders but without the *Src* and *_stored* folders. Files in those folders are stored or written in the root folder.

D.2 Files

[1t] File Types

rivt file

rivt files are written by the author. Each *rivt file* has a prefix that specifies the division label and subdivision number of the published *doc*. *rivt reports* are assembled using *doc numbers* that organize the report into divisions and subdivisions. The d*subdivision* name is taken from the name that follows the *doc number*. It may be overridden in the *API Meta function*. *Division* names are specified in the report script.

The *doc number* prefix includes a single digit capital alphanumeric division label *D*, followed by a two digit subdivision number *ss*. For example:

```
[rvDss-]file-name.py

rvC02-Beam-Loads.py
```

where the default doc name is “Beam Loads” in subdivision 2 of division C. Hyphens are stripped from file name.

Note:

The following files are read and written by *rivtlib* when processing a *rivt file*. Files are labeled by origin and folder.

origin: author input

folder: *rivt-report-folder-name*

Refer to markup for *command* details.

Run file

A *run file* is a shell or command script executed using the | RUN | command.

origin: author input

folder: *src/* - or a user created subfolder of /src

API function: *rv.R*

Insert file

An *insert file* is an image, table or text file executed using the | IMAGE | , | TABLE | , or | TEXT | command.

origin: author input

folder: *src/* - or a user created subfolder of /src

API function: *rv.I*

Value file

A *value file* is a *csv* file that defines variable values. It is imported using the | VALUE | command. Values defined by equations and a _[[VALUES]] block are written to a file with the name composed of the *doc* and *section* number. The file may be read by other *rivt files*.

Read

origin: author input

folder: *src/Values* - or a user created subfolder of /src

API function: *rv.V*

Write

origin: program output

folder: *src/Values*

API function: *rv.V*

Tool file

Tool files can be executed by | PYTHON | , | HTML | , | RST | or | LATEX | commands. They may be used for analysis and generation of text, tables, images and other formatted content. Executing a tool file may:

- define functions and classes for use in the *rivt file*
- generate images, tables and text for inclusion in the *doc*
- insert markup text into the *doc*

Read

origin: author input

folder: *src/Tools* or a user created subfolder of /src

API function: *rv.T*

Write

origin: program output

folder: *src/Tools*

API function: *rv.T*

Doc file

Each *rivt file* outputs a *doc file* with the same *doc name and number* and the selected *doc type* suffix. *Doc* files are written to the *publish folder* as text, PDF or HTML files using | PUBLISH | command.

PDF files may be prepended or appended to the *doc* using the | APPEND | command.

origin: program output

folder: *src/* - or a user created subfolder of /src

API function: *rv.D*

Report file

A *report file* is generated by the Python *report script* and is written to the designated *publish subfolder*. The *report file name* is taken from the *report folder name* unless specified in the *report script*.

origin: program output

folder: */publish/* subfolder

Report script

The *report script* is a Python file that assembles the *report*. It specifies the *docs* that should be included, whether the *rivt files* should be re-executed and other format settings.

origin: author input

folder: */publish*

Public rivt file

origin: program output

folder: */public*

A public rivt file is a copy of the *rivt file* that includes all of the sections marked as *public* in the *header*. It has the same name as the *rivt file* with an added hyphen between "rv" and the docnumber e.g. *rv-Dss-filename.py*

Log file

Log files are written to the *log folder*. They include:

- a backup of the rivt file
- a file listing the execution steps when processing the *rivt file*
- a file listing each API function call with the header.

origin: program output

folder: */logs*

[2t] Reports

A report is assembled from mutliple *docs* organized by division and subdivsion numbers. Each *doc* is a subdivision of a *report*. A typical workflow for writing *reports* is to start with a similar *report* and edit the *rivt files* and *report script* that produces the *report*.

The Python *report script* includes settings that specify assembly parameters and override defaults. These include:

- *Docs* to be included in the report
- Division names
- recompiling *docs*
- report cover page and title

provides the option to either regenerate all *docs* or to assemble the report from previously generated *docs*. Most aspects of the *report* appearance are determined at when generating *docs*.

An example script is shown below.

```
#! python

from rivtlib.rvreport import *  # noqa: F403

""" generate a rivt report

Sample report generating and config file.

Run this Python file to generate a rivt report. The report generating file
must located in /publish/subfolder where subfolder is the doc type. The
report output file name is the report title. It is written to the same
folder as the report script. Duplicate report file names are incremented.

A flag determines whether the report is assembled from existing doc files
or whether docs are regenerated prior to assembly. If a rivt or doc file is
specified for inclusion and not found a warning is given but the report is
still assembled.

A rivt report is a compilation of rivt docs organized into divisions and
subdivisions. rivt doc numbers define the organization. Default titles for
subdivisions are generated by stripping the doc number from the file name
and replacing hyphens with spaces. The default titles may be overridden in
rv.M for each rivt file.

See src/styles/rivt.ini for additional settings

"""
# ==================================
# report type
# ==================================
rptype = "pdf"  # report type [html; pdf; pdftex; text]
rvrun = False  # regenerate docs [True; False]

# ==================================
# report cover settings
# ==================================
# cover pages are located in /src
rptitle = "Solar Canopy Calculations"
rpsubtitle = "Larkspur, Ca"
rpauthor = "rhh"
rpdate = "<datetime>"
rptoc = True  # add table of contents, "" to omit
rpcover = "/publish/cover1.pdf"  # cover page, "" to omit

# ==================================
# include docs
# ==================================
# include these divisions in report
```

```
divlist = [[dv01, "Codes and Loads"],
           [dv02, "Frame"],
           [dv03, "Foundation"]]
# include these docs in report
# doc label is for reference only, may be empty string
doclist = [[rv101,"Codes"],
           [rv102,"Loads"],
           [rv201,"Frame"],
           [rv202,"Solar"],
           [rv301,"Foundation"],
           [rv302,"Walls"]]

genreport()  # noqa: F405
```

D.3 Folders

[1t] File Names

A *rivt report* is assembled from a set of *docs*. *Reports* are organized using *doc numbers*. If the *rivt file* names are:

```
rvA01-filename.py
rv105-filename.py
```

The *report numbers* used in the published report would be:

```
A.1
1.5
2.12
```

where leading zeroes are dropped and *docs* are sorted alpha-numerically into divisions and subdivisions in the *report*.

Reports are organized under a single root report folder with the prefix *rivt-*. *rivt files* are stored in the root folder and *rivt markup* file paths are relative to the roo. Resource files are stored in five subfolders:

_public
: Includes *rivt files* written by *rivtlib* intended for upload to a public repository. The prefix *rvAnn-* is changed to *rvAnn_* to avoid confusion with private files.

_publish
: Includes formatted *docs* and *reports* written by *rivtlib*.

_stored
: Includes output files written by *rivtlib* including *logs*, *values*, *hidden*, and *metadata* and *reports*

Rst_docs
: Includes formatted *docs* and *reports* written by *rivtlib*.

Src
: Includes author provided content, style and generating files for *docs* and *reports*.

An example *report* folder structure is shown below.

[3t] Report Folders

Folder Names

Top Level Folders

```
[rivt-]Report-Label/                Report Folder
    ├── [rv101-]filename1.py        rivt file
    ├── [rv102-]filename2.py        rivt file
    ├── [rv201-]filename3.py        rivt file
    ├── [rv202-]filename4.py        rivt file
    ...
    ├── [.vscode]/                  optional VSCode settings
    ├── [_public]/                  public rivt files
    ├── [_publish]/                 published docs and reports
    ├── [_stored]/                  stored files
    ├── [Rst_docs]/                 restructured text files
    ├── [Src]/                      source files
    └── README.txt                  searchable text report
```

Expanded Folders

```
[rivt-]Report-Label/                  Report Folder Name
     ├── [.vscode]/                       optional VSCode settings
     ├── [rv101-]filename1.py             rivt file
     ├── [rv102-]filename2.py             rivt file
     ├── [rv201-]filename3.py             rivt file
     ├── [rv202-]filename4.py             rivt file
     ├── [_public]/                       public rivt files
         ├── rv_101-filename1.py
         ├── rv_102-filename1.py
         ├── rv_201-filename3.py
         └── rv_202-filename4.py
     ├── [_publish]/                      published docs and reports
         ├── [docs]/                      html output
             ├── _images/
             ├── _sources/
             ├── _static/
             ├── process folders/
             ├── site folders/
             ├── rv101-filename1.html
             ├── rv102-filename2.html
             ├── rv201-filename3.html
             ├── rv202-filename4.html
         ├── [pdfdocs]/                   pdf doc output
             ├── process folders/
             ├── rv-101-filename1.pdf
             ├── rv-102-filename1.pdf
             ├── rv-201-filename3.pdf
             └── rv-202-filename4.pdf
         └── [txtdocs]/                   text output
             ├── rv-101-filename1.txt
             ├── rv-102-filename1.txt
```

```
        ├── rv-201-filename3.txt
        └── rv-202-filename4.txt
    ├── [_stored]/                      stored files from rivt
    │   ├── [Logs]/                     log files
    │   │   ├── rv101-log.txt
    │   │   └── rv102-log.txt
    │   ├── [Sect]/                     stored sections
    │   │   ├── rv202-5d.txt
    │   │   ├── rv103-4t.txt
    │   │   └── rv301-2r.txt
    │   ├── [Temp]/                     temp files
    │   │   └── rv101-label3.tex
    │   └── [Vals]/                     stored value files
    │       ├── v101-2.csv
    │       └── v102-3.csv
    ├── [Rst_docs]/                     restructured text files
    │   ├── _downloads/
    │   ├── _static/
    │   ├── _locale/
    │   ├── _templates/
    │   ├── rv101-filename1.rst
    │   ├── rv102-filename2.rst
    │   ├── rv201-filename3.rst
    │   ├── rv202-filename4.rst
    │   ├── gen-html.cmd                html generating script
    │   ├── gen-pdf.cmd                 pdf generating script
    │   ├── gen-pdftex.cmd              LaTeX generating script
    │   ├── rivt-report.py              report generating script
    │   ├── new-units.py                define new units
    │   ├── coverpage.rst               cover page template
    │   ├── logoname.png                cover page logo
    │   └── conf.py                     style paths and settings
    ├── [Src]                           source files from author
    │   ├── data/                       author created subfolder
    │   │   ├── data1.csv
    │   │   └── conc-vals.csv
    │   ├── image/                      author created subfolder
    │   │   ├── fig1.png
    │   │   └── fig2.jpg
    │   ├── output/                     author created subfolder
    │   │   ├── table1.csv
    │   │   ├── image1.png
    │   │   └── opensees1.txt
    │   ├── [Run]/                      OS commands
    │   │       ├── run1_win.cmd
    │   │       └── run1_linux.sh
    │   ├── [Tools]/                    scripts and functions
    │   │       ├── plot.py
    │   │       └── loads.py
    │   └── [Vals]/                     value files
    │       ├── steel-vals.csv
```

```
            └── plastic-vals.csv
    └── README.txt                      searchable text report
```

D.4 Configuration Files

[1] Config Files

D.4 Example

[1] rivt file

This is a small rivt file that publishes a single *doc* The report folders are not used. It has 4 API functions - two of them are published. The output is written to the terminal.

```
#! python
import rivtlib.rvapi as rv

# rv singledoc: True

# %% loads
rv.I("""Load Combinations

    ASCE 7-05 Load Effects _[T]
    ============= ================================================
    Equation No.    Load Combination
    ============= ================================================
    16-1           1.4(D+F)
    16-2           1.2(D+F+T) + 1.6(L+H) + 0.5(Lr or S or R)
    16-3           1.2(D+F+T) + 1.6(Lr or S or R) + (f1L or 0.8W)
    ============= ================================================

    | IMAGE | beam.png | Beam Geometry, .65, num

    Bending Stress Formula _[E]
    σ1 = M1 / S1 _[M]

    """)
# %% values
rv.V("""Loads and Geometry

    Beam Loads _[T]
    D_1 =: 3.8*psf | psf, kPA, 2 | joists DL
    D_2 =: 2.1*psf | psf, kPA, 2 | plywood DL
    D_3 =: 10.0*psf | psf, kPA, 2 | partitions DL
    D_4 =: 2*0.5*klf |klf, kN_m , 2 | fixed machinery  DL
    L_1 =: 40*psf | psf, kPA, 2 | ASCE7-05 LL

    | VALTABLE | beam1-v.csv | Beam Geometry, 0:0, num
```

```
    Uniform Distributed Loads
    dl_1 <=: 1.2 * (W_1 * (D_1 + D_2 + D_3) + D_4) | klf, kN_m, 2 | dead load : ASCE7-05 2.3.2  _[E]

    ll_1 <=: 1.6 * W_1 * L_1 | klf, kN_m, 2 | live load : ASCE7-05 2.3.2 _[E]

    omega_1 <=: dl_1 + ll_1 | klf, kN_m, 2 | total load : ASCE7-05 2.3.2 _[E]

    """)
# %% section properties
rv.V("""Beam Section Properties

    | PYTHON | sectprop.py | nodocstring

    section_1 <=: rectsect(10*inch, 18*inch) | in3, cm3, 2 | function: rect sect modulus _[E]

    inertia_1 <=: rectinertia(10*inch, 18*inch) | in4, cm4, 1 | function: rect moment inertia _[E]

    """)
# %% force
rv.V("""Force and Stress

        m_1 <=: omega_1 * S_1**2 / 8 | ftkips, mkN, 2 | mid-span UDL moment _[E]

        fb_1 <=: m_1 / section_1 | psi, MPA, 1 | bending stress _[E]

        fb_1 < 10*ksi | ok, not ok, blue, red, right _[E]

    """)
# %% tool
rv.T("""Metadata

    _[[PYTHON]]
    rv_metaD = {
    "authors": "rholland",
    "version": "0.7.2",
    "email": "rod.h.holland@gmail.com",
    "repo": "https://github.com/rivt-info/rivt-single-doc",
    "license": "https://opensource.org/license/mit/",
    "fork1": ["author", "version", "email", "repo"],
    "fork2": [],
    }
    _[[END]]

    """)
# %% doc
rv.D("""Publish Doc

    _[[LAYOUT]]
        logopathP: logo.png
        pdfheaderL: [docnameS, pageS, totalpageS]
        footerL: [logoO, dateO, timeO, filenameS, versionS]
    _[[END]]

    | PUBLISH | rivt | rst2pdf

    """)
```

D.5 Example

rivt files here

E.1 Collaboration

[1t] Overview

1. Public rivt files **may be shared by uploading**
   to repositories such as *GitHub*.
2. **Public *rivt files* can be found and downloaded using the**
   search interface. For public files,

   - cloned versions may be forked to create different versions.
   - pull requests may provide improvements.
   - issues and bugs may be reported.

   Public rivt files are identified with the naming convention rv-A01-filename.py, in contrast with rvA01-filename for private files. After downloading a public rivt file, use the *Batch Rename* extension to delete the hyphen after *rv* and convert it to a private rivt file.
3. Files may be collaboratively edited in real time using VSCode with the extension Visual Studio Live Share. extension.
4. Contribute to rivtlib at rivtlib-dev and documentation at rivt-info.

   - report issues and bugs
   - issue pull requests to propose changes
   - contribute to documentation

5. **Contribute extensions and scripts that improve interaction *rivt* with other**
   components of the *rivt framework*.

[2t] Git and GitHub

Git is part of the *rivt framework*. It is a free, open-source, distributed version control system designed to manage and track changes in files. It allows multiple people to work on the same project simultaneously without overwriting each other's work.

GitHub is also part of the *rivt framework*. It is a web-based platform for hosting, managing, and collaborating on code built around Git. It allows teams to work together efficiently on software projects and provides features like pull requests, issue tracking, and project management.

Each *rivt project* is typically stored in its own repository. Every user of GitHub has a personal account with essentially unlimited repositories.Free accounts provide for:

- Unlimited Repositories: You can create as many public and private repositories as you need.
- Unlimited Collaborators: There is no limit to the number of people you can work with on your repositories.
- Community Support: You have access to the GitHub Community Discussions for help.
- Core Services with Usage Limits: The free plan includes a certain amount of monthly usage for services:

  - **GitHub Actions: 2,000 minutes per month for private repositories**
    (unlimited for public repositories).
  - GitHub Codespaces: 120 core hours and 15 GB of storage per month.

GitHub Codespaces is a VSCode cloud-based development environment connected to GitHub files. It can be set up with *rivt extensions* that provide a complete rivt editing and collaboration environment in the cloud.

[3t] VSCode

VSCode is an open source IDE with a large set of extensions, including collaboration support. Collaboration is facilitated by the Visual Studio Live Share. You can start a *Live Share* session either within *CodeSpaces* in your browser, or within the *VS Code* desktop application. *CodeSpaces* is a cloud based GitHub implementation of VSCode that shares many of the same features.

The *Live Share* extension enables:

- Real-time Co-editing: Multiple participants can work on the same file simultaneously, seeing each other's cursors, selections, and edits instantly, much like in Google Docs..
- Shared Terminals and Servers: The host can share their terminal and localhost servers with guests, eliminating the need for guests to set up their own environments or dependencies.
- Navigation and Following: Participants can independently navigate the project or choose to "follow" another collaborator, keeping their editor synchronized with the leader's actions.
- File Access Control: Hosts can use a .vsls.json file to control which files and folders are visible or editable by guests, ensuring security and focus

B.4 Git

[1t] Git

GitHUb is a popular and widely used cloud and version control system based on the Git program that may be downloaded here.

E.3 GitHub Search

[1t] GitHub Global Search

This page is a search interface for **rivt** files on GitHub. The search covers the full text of the **rivt** README.txt file in the *repo* directory. The README file contains the *rivt report*. For advanced searching use the GitHub interface.

[S] Search GitHub <Ctrl+Enter>

[C] Clear input <Ctrl+R>

Example: solar+steel+frame is passed to GitHub as rivt+solar+steel+frame

[2t] GitHub Organization Search

Restrict search to the following comma separated GitHub organizations:

E.2 Public rivt files

[1t] Overview

A feature of *rivt* is fine-grained control over public sharing of all or parts of a rivt file. *Public rivt files* are generated by *rivtlib* on a section by section basis defined by the API header substring. Each API function defines a section and has a public/private setting in the header. Sections are private by default (see *:doc: <rvF01-quickref>*).

```
rv._("""Section Label | doc;stored, private;public, section;merge

     Content substring indented 4 spaces

    ...

    """)
```

Sections are written to a rivt file in the */public* folder with a file name composed as:

```
rivt file name

    rvAnn-filename.py

modified with an additional hyphen

    rv-Ann-filename.py
```

The public folder may be uploaded to a GitHub repo for public sharing. When downloaded the extra hyphens may be removed with a batch remove command (e.g. in VSCode).

[2t] Public Folder

For each rivt file that is published a corresponding *public rivt file* is written to the *public* folder. The default contents of the file are the API rivt string headers. If the header parameter *public* is included in the header, the rivt string content is also written to the *public rivt file*.

```
[rivt]-Report-Label/                Report Folder Name
    ├── [rv101-]filename1.py        | rivt file
    ├── [rv102-]filename2.py        | rivt file
    ├── [rv201-]filename3.py        | rivt file
    ├── [rv202-]filename4.py        | rivt file

    ...

    ├── [public]/                   || public rivt files
        ├── rv-101-filename1.py
        ├── rv-102-filename1.py
        ├── rv-201-filename3.py
        └── rv-202-filename4.py
```

The *public folder* can be uploaded to a public repository and updated with Git as the project develops.

[3t] Example

F.1 Quick Ref

[1t] API Summary

API functions

Function	Name	Purpose
rv.R (rS)	Run	Run shell commands
rv.I (rS)	Insert	Insert static sources
rv.V (rS)	Values	Calculate values
rv.T (rS)	Tools	Python and markup scripts
rv.D (rS)	Doc	Publish docs
rv.S (rS)	Skip	Skip section (comments and debugging)
rv.X (rS)	Exit	Exit rivt (debugging)

where **rS** is a *rivt string*. The first line of a *rivt string* (rS) is the header substring.

```
rv.I("""A New Section | private, doc, section

    Content text
    ...

    """)
```

The default setting for each API is listed **in bold** in the table below. Default settings in the *header substring* do not need to be specified.

API	private;public	doc;stored	section;merge
rv.R	**private**;public	**stored**;doc	**merge**;section
rv.I	**private**;public	**doc**;stored	**section**;merge
rv.V	**private**;public	**doc**;stored	**section**;merge
rv.T	**private**;public	**stored**;doc	**merge**;section
rv.S	**private**;public	**stored**	**merge**
rv.D	**public**	**stored**	**merge**

API	private;public	doc;stored	section;merge
rv.X	•	•	•

[2t] Line Tags

Format a line of text

API Scope	Line Tag	Description
rv.I	text **_[C]**	[2t] Center text
rv.I	text **_[R]**	[3t] Right justify text
rv.I	**text math _[M]**	[4t] Text math
rv.I	**LaTeX math _[L]**	[5t] LaTeX math
rv.I	text **_[#]** text	[6t] Endnote number
rv.I	text **_[G] glossary term |** text	[7t] Glossary link
rv.I	text **_[S] label, section |** text	[8t] Section link
rv.I	text **_[D] label, file name |** text	[9t] Doc link
rv.I	text **_[U] label, url |** text	[10t] URL link
rv.V, I	text **_[V] var_name |** text	[11t] Variable value
rv.V, I	**text _[E]**	[12t] Equation number
rv.V, I	**title _[T]**	[13t] Table number
rv.V, I	**caption _[F]**	[14t] Figure number
rv.V, I	**_[P]**	[15t] New Page
rv.V, I	**##** text	nonprinting comment

[3t] Block Tags

Format a block of text

API Scope	Block Tag	Description
rv.R	**_[[SHELL]]** process parameters	[2t] Shell script
rv.I	**_[[INDENT]]** spaces (4 default)	[3t] Indent text

API Scope	Block Tag	Description
rv.I	**_[[ITALIC]]** spaces (4 default)	[4t] Indent italic
rv.I	**_[[ENDNOTES]]** optional label	[5t] Endnotes
rv.I	**_[[TEXT]]** optional language	[6t] Text
rv.I	**_[[TOPIC]]** topic	[7t] Topic
rv.I	**_[[BOX]]** label	[8t] Box
rv.V, T	**_[[PYTHON]]** namespace	[9t] Python
rv.D	**_[[METADATA]]** label	[11t] Metadata
rv.D	**_[[LAYOUT]]** label	[12t] Layout block
all	**_[[END]]**	[13t] End block

[4t] Command Summary

format files and calculations

API Scope	Command	Description
rv.R	**SHELL** | rel path | os, wait	[2t] Shell file
rv.I	**TEXT** | rel path | language	[3t] Text file
rv.V, I	**TABLE** | rel path | title,width,rows,align,head,num	[4t] Table file
rv.V, I	**IMAGE** | rel path | caption, scale, number	[5t] Image file
rv.V, I	**IMAGE2** | rel pth1, rel pth2 | cap1,cap2,sca1,sca2,num	[6t] Adjacent images
rv.V	**VALTABLE** | rel path | title, rows, number	[7t] Valtable file
rv.V	a **==:** 1*IN | unit1, unit2, decimal | label	[8t] Define value
rv.V	c **<=:** expression | unit1, unit2, decimal | label, number	[9t] Assign value
rv.V	c **:=:** expression | unit1, unit2, decimal | label, number	[9t] Function value

API Scope	Command	Description
rv.V	a **<** c | decimal | text1, text2, color1, color2	[10t] Compare values
rv.V, T	**PYTHON** | rel path | namespace	[11t] Python file
rv.T	**MARKUP** | rel path | type	[12t] Markup file
rv.D	**ATTACHPDF** | rel path | place, title	[13t] Attach PDF
rv.D	**PUBLISH** | doc title | type	[14t] Publish doc

default paths for single docs

See here for the folder structure. If files are in the default path only the file name needs to be provided.

Command	Default Path
| SHELL |	**folder root**
| TEXT |	**folder root**
| TABLE |	**folder root**
| IMAGE |	**folder root**
| IMAGE2 |	**folder root**
| VALTABLE |	**folder root**
| PYTHON |	**folder root**
| MARKUP |	**folder root**
| ATTACHPDF |	**folder root**
| PUBLISH |	**folder root**

Default command paths for report

See here for the folder structure. If files are in the default path only the file name needs to be provided.

Command	Default Path
| SHELL |	**/Files/Scripts/**
| TEXT |	**/Files/Data/**

Command	Default Path
| TABLE |	**/Files/Data/**
| IMAGE |	**/Files/Image/**
| IMAGE2 |	**/Files/Image/**
| VALTABLE |	**/Files/Data/** [1]
| PYTHON |	**/Files/Scripts/**
| MARKUP |	**/Files/Scripts/**
| ATTACHPDF |	**/Files/Attach/**
| PUBLISH |	depends on file type

[1] use /stored/data/filename to read values defined in the rivt file

[5t] Folders

Single doc folders

```
[rivt-]single-doc-label/                Single doc Folder
    ├── [rv101-]filename.py             |  rivt input file
    ├── multiple source files           |  data, image and function files
    ├── addunits.py                     |  define new units
    ├── rv-101-log.txt                  || log file
    ├── rv-101-docname.py               || public rivt file
    ├── README.txt                      || searchable text doc
    ├── [.vscode]/                      |  optional VSCode settings
    ├── [Rstdocs]/                      |  rst style input
        ├── _downloads/                 |  external doc files
        ├── _static/                    |  style files
        ├── _locale/                    |  style files
        ├── _templates/                 |  style files
        ├── coverpage.rst               |  pdf cover page template
        ├── logoname.png                |  cover page logo
        ├── conf.py                     |  style settings
        └── rv101-filename.rst          || intermediate rst file
    ├── [pdfdocs]/                      || pdf output
        ├── process folders/               pdf working files
        └── rv101-filename.pdf             pdf doc
    ├── [htmldocs]/                     || html output
        ├── process folders/               html process folders
        ├── site folders/                  html site folders and files
        └── rv101-filename.html            html doc
    ├── [latexdocs]/                    || latex output
        ├── latexstyle.sty              |  latex pdf style file
        ├── process files/                 latex process files
        └── rv101-filename.pdf             pdf doc from LaTeX
```

```
    ├── [textdocs]/                    || text output
        └── rv101-filename.txt            text doc
```

Reports - Top Level Folders

```
[rivt-]Report-Label/                Report Folder
    ├── [rv101-]filename1.py        | rivt file
    ├── [rv102-]filename2.py        | rivt file
    ├── [rv201-]filename3.py        | rivt file
    ├── [rv202-]filename4.py        | rivt file
    ...
    ├── [Files]/                    |  source and style files from author
    ├── [publish]/                  || doc and report files
    ├── [stored]/                   || rivt stored files
    └── README.txt                  || searchable text report
```

Reports - Expanded Folders

```
[rivt-]Report-Label/                        Report Folder Name
    ├── [rv101-]filename1.py                |  rivt file
    ├── [rv102-]filename2.py                |  rivt file
    ├── [rv201-]filename3.py                |  rivt file
    ├── [rv202-]filename4.py                |  rivt file
    ├── [.vscode]/                          |  optional VSCode settings
    ├── [Files]/                            |  files from authors
        ├── [Rstdocs]/                      |  intermediate rst files
            ├── _downloads/                 |  external doc files
            ├── _static/                    |  style files
            ├── _locale/                    |  style files
            ├── _templates/                 |  style files
            ├── rv101-filename1.rst         || intermediate rst file
            ├── rv102-filename2.rst
            ├── rv201-filename3.rst
            ├── rv202-filename4.rst
            ├── coverpage.rst               |  pdf cover page template
            ├── logoname.png                |  cover page logo
            └── conf.py                     |  style paths and settings
        ├── [Data]/                         |  data files
            ├── data1.csv
            └── conc-vals.csv
        ├── [Image]/                        | image files
            ├── fig1.png
            └── fig2.jpg
        ├── [Scripts]/                      | script and shell files
            ├── shell1.cmd
            ├── shell1.csv
            ├── plot.py
            ├── addunits.py
            └── opensees1.cmd
        └── [Attach]/                       | pdf attachments
            ├── attach1.pdf
```

```
        └── attach2.pdf
    ├── [publish]/                      || published docs and reports
        ├── [htmldocs]/                 || html output
            ├── process folders/           html process folders
            ├── site folders/              html site folders and files
            ├── rv101-filename1.html
            ├── rv102-filename2.html
            ├── rv201-filename3.html
            ├── rv202-filename4.html
        ├── [latexdocs]/                || latex output
            ├── process folders/        || latex process folders
            ├── latexstyle.sty          |  pdf style file
            ├── rv101-filename.rst      || intermediate rst file
            ├── rv101-filename1.rst
            ├── rv102-filename2.rst
            ├── rv201-filename3.rst
            ├── rv202-filename4.rst
            ├── rv-101-filename1.pdf    || latex pdf doc output
            ├── rv-102-filename1.pdf
            ├── rv-201-filename3.pdf
            └── rv-202-filename4.pdf
        ├── [pdfdocs]/                  || pdf doc output
            ├── process folders/           pdf working files
            ├── rv-101-filename1.pdf
            ├── rv-102-filename1.pdf
            ├── rv-201-filename3.pdf
            └── rv-202-filename4.pdf
        ├── [publicfiles]/              || public rivt files
            ├── rv-101-filename1.py
            ├── rv-102-filename1.py
            ├── rv-201-filename3.py
            └── rv-202-filename4.py
        └── [textdocs]/                 || text output
            ├── rv-101-filename1.txt
            ├── rv-102-filename1.txt
            ├── rv-201-filename3.txt
            └── rv-202-filename4.txt
    ├── [stored]/                       || stored files from processing
        ├── [logs]/                     ||log files
            ├── rv101-log.txt
            └── rv102-log.txt
        ├── [sect]/                     || excluded sections
            ├── rv202-5d.txt
            ├── rv103-4t.txt
            └── rv301-2r.txt
        └── [data]/                     || rivt generated data
            ├── table1.csv                 stored script output
            ├── image1.png                 stored images
            └── v102-3.csv                 rivt value table output
    └── README.txt                      || searchable text report
```

[6t] Project requirements

The minimum Python version is *3.14*. A *rivt* project installation includes Python packages for:

- document formatting
- numerical analysis
- symbolic processing
- visualization

Dependency	description
“pyzo>=4.20.0”	lightweight IDE
“pyside6>=6.10.1”	QT bindings
“fastcore>=1.8.16”	code simplification
“tabulate>=0.9.0”	format tables
“pillow>=11.2.1”	image processing
“matplotlib>=3.10.1”	data visualization
“sympy>=1.13.3”	symbolic analysis
“numpy>=2.2.5”	numerical analysis
“scipy>=1.16.3”	numerical analysis
“pandas>=2.2.3”	data analysis
“docutils>=0.21.2”	reStructuredText processing
“ipython>=8.16.2”	interactive Python shell
“ipykernel>=6.28.1”	Jupyter kernel for Python
“reportlab>=4.4.0”	PDF generation without LaTeX
“rst2pdf>=0.103.1”	PDF generation without LaTeX
“pypdf>=1.0.3”	PDF manipulation
“Sphinx>=8.2.3”	HTML generation
“pydata-sphinx-theme>=0.16.1”	HTML generation
“sphinx-copybutton>=0.5.2”	HTML generation
“sphinx_design>=0.6.1”	HTML generation
“sphinx-favicon>=1.0.1”	HTML generation
“sphinx-togglebutton>=0.3.2”	HTML generation
“sphinxcontrib-applehelp>=2.0.0”	HTML generation

Dependency	description
“sphinxcontrib-devhelp>=2.0.0”	HTML generation
“sphinxcontrib-email>=0.3.6”	HTML generation
“sphinxcontrib-htmlhelp>=2.1.0”	HTML generation
“sphinxcontrib-jquery>=4.1”	HTML generation
“sphinxcontrib-jsmath>=1.0.1”	HTML generation
“sphinxcontrib-qthelp>=2.0.0”	HTML generation
“sphinxcontrib-serializinghtml>=2.0.0”	HTML generation

[7t] VSCode settings

Workspace extension and other settings are stored in the *.vscode* folder and can be included and moved as part of a *rivt report*. Settings that affect the *rivt* environment incldue:

- rivt.code-snippets
- settings.json (extension settings)
- tasks.json
- launch.json

Global settings can be exported and imported using VSCode Profiles. VSCode *rivt* extensions and settings may be installed from this link.

Extensions	description
BUTTONS	—
tombonnike.vscode-status-bar-format-toggle	format button
gsppvo.vscode-commandbar	command buttons
AdamAnand.adamstool	command buttons
nanlei.save-all	save all button
Ho-Wan.setting-toggle	toggle settings
yasukotelin.toggle-panel	toggle panel
fabiospampinato.vscode-commands	user command buttons
jerrygoyal.shortcut-menu-bar	menu bar
EDITING	—
henryclayton.context-menu-toggle-comments	toggle comments

Extensions	description
TroelsDamgaard.reflow-paragraph	wrap paragraph
streetsidesoftware.code-spell-checker	spell check
jmviz.quote-list	quote elements in a list
njpwerner.autodocstring	insert doc string
oijaz.unicode-latex	unicode symbols from latex
jsynowiec.vscode-insertdatestring	insert date string
janisdd.vscode-edit-csv	csv editor
VIEWS	—
GrapeCity.gc-excelviewer	excel viewer
SimonSiefke.svg-preview	svg viewer
tomoki1207.pdf	pdf viewer
RandomFractalsInc.vscode-data-preview	data viewing tools
Fr43nk.seito-openfile	open file from path
vikyd.vscode-fold-level	line folding tool
file-icons.file-icons	icon library
tintinweb.vscode-inline-bookmarks	inline bookmarks
MANAGEMENT	—
alefragnani.project-manager	folder, project management
Anjali.clipboard-history	clipboard history
dionmunk.vscode-notes	notepad
hbenl.vscode-test-explorer	test explorer
mightycoco.fsdeploy	save file to second location
lyzerk.linecounter	count lines in files
sandcastle.vscode-open	open files in default app
zjffun.snippetsmanager	snippet manager
spmeesseman.vscode-taskexplorer	task explorer
GITHUB	—
GitHub.codespaces	run files in codespaces

Extensions	description
GitHub.remotehub	run remote files
ettoreciprian.vscode-websearch	search github within VSCode
donjayamanne.githistory	git history
MichaelCurrin.auto-commit-msg	git auto commit message
github.vscode-github-actions	github actions
GitHub.vscode-pull-request-github	github pull request
k9982874.github-gist-explorer	gist explorer
vsls-contrib.gistfs	gist tools
PYTHON	—
ms-python.autopep8	python pep8 formatting
ms-python.isort	python sort imports
donjayamanne.python-environment-manager	python library list
ms-python.python	python tools
ms-python.vscode-pylance	python language server
ms-toolsai.jupyter	jupyter tools
ms-toolsai.jupyter-keymap	jupyter tools
ms-toolsai.jupyter-renderers	jupyter tools
ms-toolsai.vscode-jupyter-cell-tags	jupyter tools
ms-toolsai.vscode-jupyter-slideshow	jupyter tools
LANGUAGES	—
qwtel.sqlite-viewer	sqlite tools
RDebugger.r-debugger	R tools
REditorSupport.r	R tools
ms-vscode-remote.remote-wsl	windows linux tools
James-Yu.latex-workshop	latex tools
lextudio.restructuredtext	restructured text tools
trond-snekvik.simple-rst	restructured syntax
yzane.markdown-pdf	markdown to pdf

Extensions	description
yzhang.markdown-all-in-one	markdown tools

Snippets/Keys	description
run	API Run function
ins	API Insert function
val	API Values function
too	API Tools function
wri	API Write function
alt+q	rewrap paragraph with hard line feeds (80 default)
alt+p	open file under cursor
alt+.	select correct spelling under cursor
alt+8	insert date
alt+9	insert time
ctl+1	focus on first editor
ctl+2	focus on next editor
ctl+3	focus on previous editor
ctl+8	focus on explorer pane
ctl+9	focus on github pane
ctl+alt+x	reload window
ctl+alt+[	reload window
ctl+alt+]	unfold all code
ctl+alt+u	unfold all code
ctl+alt+f	fold code level 2 (rivt sections visible)
ctl+alt+a	fold code - all levels
ctl+alt+t	toggle local fold
ctl+alt+e	toggle explorer sort order
ctl+alt+s	toggle spell check
ctl+alt+g	next editor group

Snippets/Keys	description
ctl+shift+u	open URL under cursor in browser
ctl+shift+s	open GitHub README search for rivt
ctl+shift+a	commit all
ctl+shift+z	commit the current editor
ctl+shift+x	post to remote

Keystroke	Description
alt+q	rewrap paragraph with hard line feeds (80 default)
alt+p	open file under cursor
alt+.	select correct spelling under cursor
alt+8	insert date
alt+9	insert time
ctl+1	focus on first editor
ctl+2	focus on next editor
ctl+3	focus on previous editor
ctl+8	focus on explorer pane
ctl+9	focus on github pane
ctl+alt+x	reload window
ctl+alt+u	unfold all code
ctl+alt+f	fold code level 2 (rivt sections visible)
ctl+alt+a	fold code - all levels
ctl+alt+t	toggle local fold
ctl+alt+e	toggle explorer sort order
ctl+alt+s	toggle spell check
ctl+alt+g	next editor group
ctl+shift+u	open URL under cursor in browser
ctl+shift+s	open GitHub rivt README search
ctl+shift+a	commit all

Keystroke	Description
ctl+shift+z	commit current editor
ctl+shift+x	post to remote

A.2 Terms

[1t] rivt Terms

block tag : block tags

A block tag formats a block of text that begins with **_[[TAG]]** and terminates with **_[[Q]]**.

command : commands

rivt commands read and write external files and assign values to variables. They start in the initial columns with a vertical bar (|) followed by the file path, name and parameters. The path is relative to the rivt report root folder. If the metadata variable *rv_localB* is set to true resources will be read from and written to the local *rivt file* folder.

```
| COMMAND | relative path | parameters
```

content : content text : content substring

The text in a *rivt string* that follows the *header substring*. The text is utf-8 (Unicode Transformation Format - 8-bit), a character encoding standard that is a superset of ASCII, handles text from any language, and is constant across all platforms.

division

A group of related rivt files organized in a report and labeled in the rivt file name. Optionally organized in a division folder.

doc : docs

A formatted document file output using the rivtlib package. A doc is a text, PDF or HTML file with the default name of the *rivt file* and file type suffix.

doc number

Prefix of a rivt and doc file name and used to organize a report. It has the form *rvDss-filename.py* where D is a capital alphanumeric division number and ss is the subdivision number.

header : headers : header text : header substring

The first line of a *rivt string* that includes a section title, followed by comma separated parameters.

line tag : line tags

A line tag has the form **_[TAG]** and formats a line of text.

report : reports : rivt report : rivt reports

A group of compiled *docs* organized by rivt file number.

report folder : report folders

The folder structure for producing a report is described here.

report script

A Python script that assembles *docs* into a *report*.

rivt

An open source Python project that includes rivtlib and approximately two dozen Python packages. When *rivt* is installed docs and reports may be edited and published using a text editor.

rivt doc : rivt docs

A formatted document file output from a *rivt file* using the rivtlib package. A doc is a text, PDF or HTML file with the default name of the *rivt file* and file type suffix.

rivt file : rivt files

A Python file that imports *rivtlib* and includes *rivt markup*.

rivt file number

Prefix of rivt file name used to organize a report (rvDss-). It has the form *rvDss-filename.py* where D is a capital alphanumeric division number and ss is the subdivision number.

rivt folder

A folder containing a rivt file and all of its resources in the same folder. Generally used

rivt framework : framework

The recommended *rivt-framework* includes the following separately installed programs:

1. *VSCode <https://code.visualstudio.com/>* and *VSCode extensions <https://marketplace.visualstudio.com/vscode>*. *rivt* extensions and snippets are here.
2. *Git <https://git-scm.com>* and set up a *GitHub <hhttps://github.com>*
3. *LaTeX <https://www.tug.org/texlive/>*
4. *QCAD <https://qcad.io/en/>*

rivt log file

rivt file execution log written to the *log folder* as *rvDss-log.txt*.

rivt markup

A light weight markup that wraps restructuredText and outputs reports in text, PDF and HTML. Markup is included in string arguments to rivt API functions.

rivt report folder

A folder containing multiple rivt files to be organized in a report and resources organized in subfolders

rivt string

Triple quoted string argument to an API function.

rivtlib

Open source Python package that generates docs and reports from a rivt file

rvspace

rivt alias name for the global rivt file namespace. Other user namespaces may be specified

section parameter : section parameters

Comma separated parameters in a *header* that specify the section processing.

single doc : single docs

A single doc is a rivt file that publishes a doc that is not part of a report and is used for quick docs that only require limited formatting.

It is specified in a comment variable directly after the *rivtlib* import statement.

```
# rv singledoc = True
```

The default setting is False. The rivt file folder is used for source and output files. A folder structure is not used.

[2t] Python Terms

docutils

A Python package that processes restructured text files into HTML, LaTeX, and other formats.

namespace

Provides scope for functions and variables.

Python : python

A programming language that lets you work quickly and integrate systems more effectively

restructuredtext : restructuredtext markup

A lightweight markup language designed to be processed by document software including docutils, Sphinx and rivt.

[3t] GitHub and VSCode Terms

fork : forked

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. Most commonly, forks are used to either propose changes to someone else's project or to use someone else's project as a starting point for your own idea.

Git : git

A version control system that lets you manage and keep track of your source code history.

IDE : ide

Integrated Development Environment. A software application that provides comprehensive facilities to computer programmers for software development. An IDE typically includes a code editor, build automation tools, and a debugger. *rivt extensions* have been developed for VSCode. Other IDEs include PyCharm, Spyder, and JupyterLab.

profile

Allows VSCode users to customize their environment for different workflows, projects, or tasks. This feature provides a way to manage distinct configurations of settings, extensions, keyboard shortcuts, snippets, and tasks.

repo : repository

a storage location for software packages

rivt Codespace : rivt codespace

A Codespace set up with extensions, snippets and keyboard shortcuts for editing and publishing *rivt docs*.

Index

S